

Exploring the Relationship Between User Ratings and Box Office Success

Katie Northrup

ISTA 322

7 May 2025

Data:

This project utilizes two primary data sources: the MovieLens (ml-latest-small) dataset and the TMDb (The Movie Database) API. Together, these sources allow for the analysis of user preferences, tagging behavior, and financial performance of films.

1. MovieLens (ml-latest-small)

URL: <https://grouplens.org/datasets/movielens/>

Curated by the GroupLens research team at the University of Minnesota, this dataset contains user-generated movie ratings, free-form tags, and metadata such as movie titles and genres. It is commonly used for building and evaluating recommendation systems and is particularly well-suited for relational modeling due to its clean, structured format.

2. The Movie Database (TMDb) API

URL: <https://www.themoviedb.org/documentation/api>

TMDb provides a wide array of movie metadata, including financial figures (budget and revenue), production companies, release dates, and additional identifiers such as IMDb IDs. The API is openly accessible with registration and offers detailed information that is not present in the MovieLens dataset.

Available and Utilized Data

The following files from the MovieLens dataset were used:

- **movies.csv:** Contains movie titles and genres.
- **ratings.csv:** Provides user ratings for movies along with timestamps.
- **tags.csv:** Includes tags applied by users to movies, offering qualitative insight into viewer sentiment.
- **links.csv:** Provides mappings from MovieLens movie IDs to IMDb and TMDb IDs, enabling external enrichment.

From the TMDb API, the following fields were extracted:

- **budget:** The production budget of the film (in USD).
- **revenue:** Total box office earnings (in USD).
- **production_companies:** Names of the companies involved in production.
- **Additional metadata** such as `tmdb_id` and `imdb_id` used for validation and joins.

Data Integration and Workflow

The pipeline starts by loading and prepping the MovieLens CSV files into pandas DataFrames. Movie titles are cleaned up using regular expressions to pull out the release year, which is then moved into its own column. This helps ensure smoother matching with external data sources.

Next, the script uses TMDb IDs from the `links.csv` file to pull financial info from the TMDb API. For each valid ID, it grabs data like budget, revenue, and production companies, which gets stored in a separate financials table.

Finally, all the data is loaded into a local MySQL database using SQLAlchemy, creating a well-organized relational dataset that's ready for complex queries and visualizations.

Cleaning and Normalization Challenges

1. Title Format Variability

Many movie titles in the MovieLens dataset include the release year in parentheses. While that's helpful for context, it needs to be cleaned up for consistent analysis and database joins. We used regular expressions to separate the title and release year into distinct fields. Titles that couldn't be reliably parsed were dropped.

2. Missing or Invalid IDs

Some movies didn't have valid TMDb IDs, which are necessary for pulling financial data. To avoid failed API calls, we filtered out any records without a proper `tmdbId`, which narrowed down the list of movies eligible for enrichment.

3. API Rate Limits and Latency

The TMDb API comes with rate limits and can be slow when querying a large number of movies. To keep things efficient and consistent, we limited the enrichment to the first 100 movies with valid TMDb IDs, sorted by `movie_id`. This keeps the process reproducible and within API usage limits.

4. Incomplete Financial Data

Even when movies had valid TMDb entries, budget and revenue data were often missing or set to zero. This is especially true for older titles, indie films, or international releases where data may not be available. We kept these entries as-is to reflect the real-world nature of financial data gaps.

5. Reproducibility and Sampling

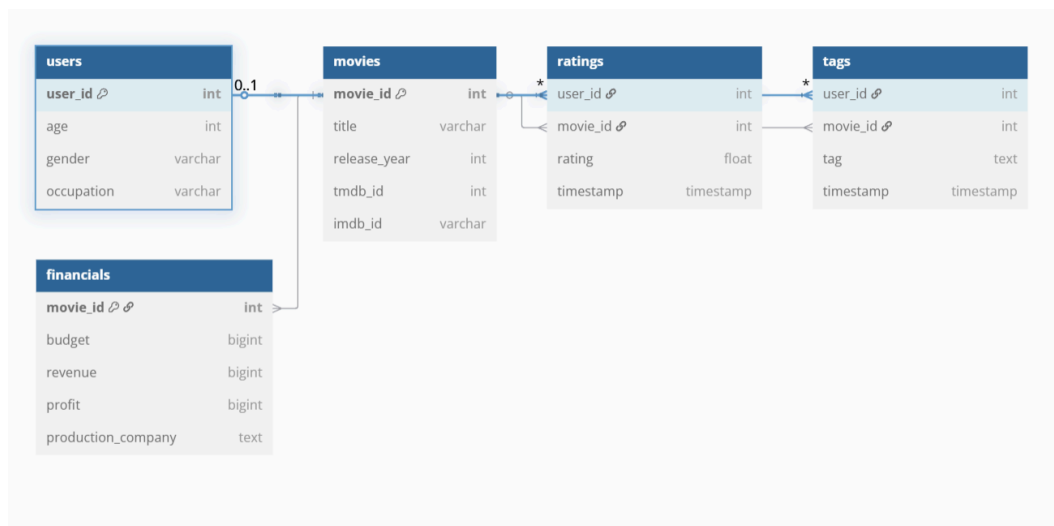
Earlier versions of the pipeline used `.sample(20)` to randomly select movies for API enrichment, but that led to inconsistent results. We switched to `.sort_values('movie_id').head(100)` to ensure the same movies are selected each time, improving reproducibility.

Relational Database Schema:

Schema Design

The relational schema was designed with normalization and extensibility in mind. Each core entity, users, movies, tags, and financial data, is stored in its own table. Relationships between entities are defined using foreign keys to ensure referential integrity and support advanced analytical queries.

ERD



Relationships

- The movies table acts as the central hub.
- ratings and tags tables form many-to-many relationships between users and movies.
- financials has a one-to-one relationship with movies, using movie_id as both primary and foreign key.

This structure ensures that user-generated and external data are kept modular while maintaining tight integration through well-defined keys.

Queries and Plots:

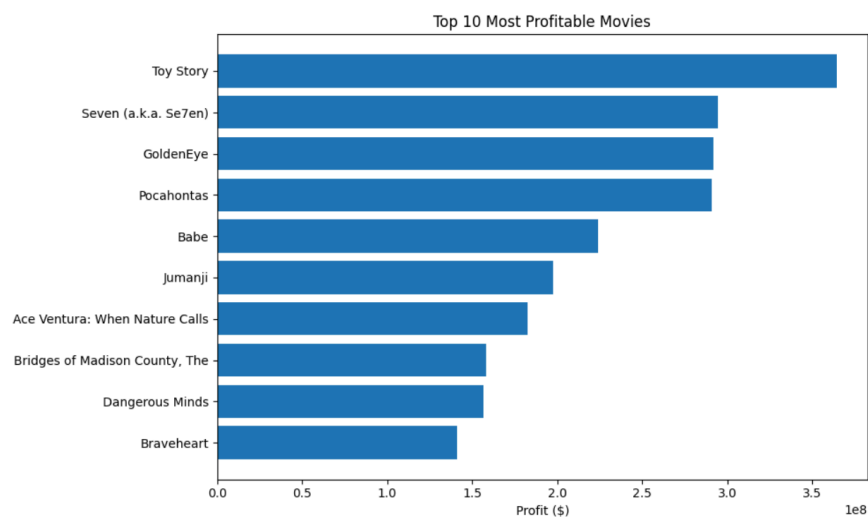
To show the benefits of combining user-generated data with external financial metadata, we ran a series of analytical SQL queries. These queries explore different angles of film performance, like

profitability, popularity, and audience sentiment. We used Python's Matplotlib to create visualizations that make the insights easier to interpret and highlight key patterns quickly.

Query 1: Most Profitable Movies

Code -

```
SELECT m.title, f.profit
FROM financials f
JOIN movies m ON f.movie_id = m.movie_id
ORDER BY f.profit DESC
LIMIT 10;
```



This query identifies the movies that generated the highest profits, calculated as the difference between revenue and budget. It serves to highlight which films achieved the greatest financial return, irrespective of critical or audience reception.

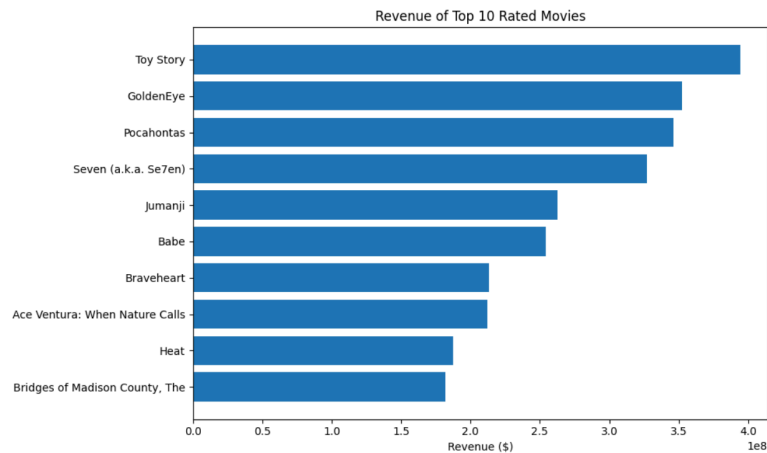
A horizontal bar chart was used to present profit figures by title. The chart revealed that a small number of films accounted for a large portion of the total profit, consistent with the “blockbuster effect” seen in the film industry. This visualization provides an entry point for exploring genre or studio characteristics of financially successful movies.

Query 2: Top Grossing Movies and Average Ratings

Code -

```
SELECT m.title, ROUND(AVG(r.rating), 2) AS avg_rating, f.revenue
FROM ratings r
JOIN movies m ON r.movie_id = m.movie_id
JOIN financials f ON m.movie_id = f.movie_id
```

```
GROUP BY m.movie_id, m.title, f.revenue
ORDER BY f.revenue DESC
LIMIT 10;
```



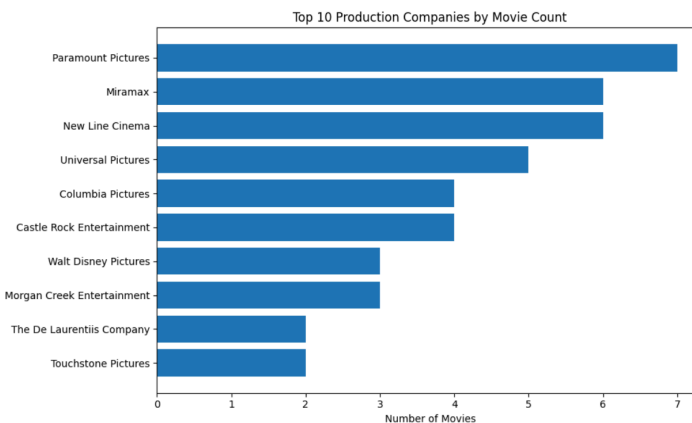
This query analyzes the relationship between commercial success (revenue) and user sentiment (average rating). It determines whether high box office earnings are aligned with positive audience reviews.

A bar chart was used to compare revenue alongside ratings. The analysis showed mixed alignment: while some high-grossing films also had strong ratings, others achieved financial success despite average or below-average audience reception. This confirms that financial success is not always dependent on viewer satisfaction and may be influenced by marketing, franchise appeal, or timing of release.

Query 3: Production Companies by Output

Code -

```
SELECT production_company, COUNT(*) AS movie_count
FROM financials
GROUP BY production_company
ORDER BY movie_count DESC
LIMIT 10;
```



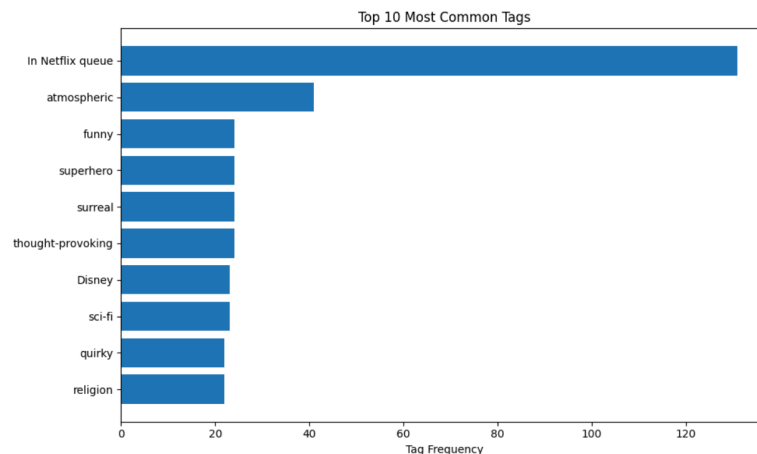
This query ranks production companies by the number of films they have in the dataset. It serves to identify the most prolific studios within the sampled set.

The bar chart highlighted a few studios that were significantly more active than others. Although the dataset was limited to 20 enriched movies, this still provided insight into studio presence and helped to contextualize which companies were responsible for multiple entries. This is particularly useful when analyzing profitability and quality trends by studio.

Query 4: Most Frequent User Tags

Code -

```
SELECT tag, COUNT(*) AS frequency
FROM tags
GROUP BY tag
ORDER BY frequency DESC
LIMIT 10;
```



To uncover viewer perception and recurring themes in how users describe and categorize films. Tags offer qualitative context that extends beyond quantitative ratings.

The top tags charted included common genres like “action” and “comedy” as well as more niche descriptors such as “cult classic” or “based on a true story.” These tags reflect the emotional or thematic lens through which viewers engage with content and provide useful signals for personalization or content discovery.

Query 5: Highest-Rated Movies with Minimum 10 Ratings

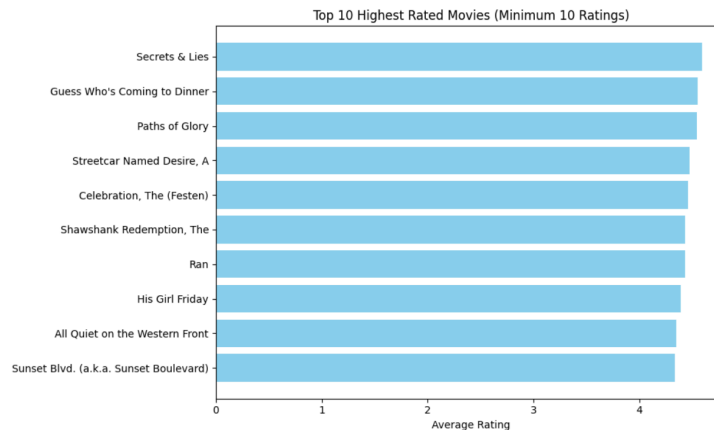
Code -

```
SELECT
  m.title,
```

```

ROUND(AVG(r.rating), 2) AS avg_rating,
COUNT(*) AS num_ratings
FROM ratings r
JOIN movies m ON r.movie_id = m.movie_id
GROUP BY m.movie_id, m.title
HAVING num_ratings >= 10
ORDER BY avg_rating DESC
LIMIT 10;

```



To rank films based on user satisfaction, with the condition that each film must have a minimum of 10 ratings. This threshold helps avoid skewed averages due to small sample sizes.

A horizontal bar chart showed the highest-rated films among those with sufficient user engagement. The result revealed several critically acclaimed or cult-favorite titles. This type of analysis is especially valuable for recommendation systems, as it identifies universally well-received content.

Executive Summary and Conclusion:

This project highlights how combining user-generated data with external metadata can lead to meaningful, insightful analysis. We built a normalized SQL schema using data from MovieLens and TMDb, and developed a pipeline that handled data cleaning, transformation, API integration, and loading into a relational database.

Even with some limitations, like TMDb rate limits, patchy financial data, and a smaller enrichment sample, the project still delivered structured, reproducible results. Our analytical queries uncovered trends in profitability, audience preferences, and studio output. Interestingly, the most profitable films didn't always match the highest-rated ones, showing the gap between critical reception and box office success.

Overall, the project shows a scalable, flexible way to build analytics infrastructure using open data. There's plenty of room to grow, future improvements could include enriching more movies via the API, adding large-scale caching, or incorporating deeper user data for demographic analysis. This setup could also lay the groundwork for machine learning, recommendation engines, or predictive tools in the entertainment space.